

縦スクロール弾幕シューティング 企画書

作品名 Spell Burst プログラム技術解説資料

制作者 田中 壮 制作形態 個人制作

1. 作品概要および基本構造

本作品 Spell Burst は、圧倒的な密度を誇る敵弾幕をプレイヤーの精密な立ち回りとリアルタイムの成長要素によって切り開く爽快縦スクロール弾幕シューティングゲームです。市販エンジンに依存せず、基盤となるゲームループや各種管理機構を純粋なC++によって構築することで、高負荷な弾幕生成や演出処理においても高い応答性と安定した動作フレームレートを実現しています。

開発環境	C++、DxLib 描画機能のみ使用、Visual Studio 2022
開発期間・形態	約1ヶ月間 個人制作 全般ロジック・弾幕アルゴリズム・ボスステート・リソース制御を担当
ゲーム要素	特徴の異なる自律型3タイプの自機、ゲージ消費型の広範囲必殺技スペルカード、敵撃破時の経験値によるリアルタイムレベルアップ機能、ボスの残り体力に連動した多段フェーズ移行システム

近接特化型自機

射程は短いものの超高威力を誇るブレード攻撃を搭載。敵の弾幕間隙へ危険を冒して肉薄するリスクとリターンを重視した機体。

波動範囲特化型自機

画面広範囲をカバーする拡散波動ショットを照射。雑魚敵の集団を即座に掃討し、画面上の安全圏を確保しやすい初心者向け機体。

追尾特化型自機

誘導性能を持つホーミング弾を常時展開。自機の回避行動に全精力を集中させつつ、確実に敵へダメージを与え続ける安定型機体。

2. プログラムの全体設計とアーキテクチャ

特定のゲームエンジンのフレームワークに依拠せず、グラフィックや音声の低レイヤー描画処理として DxLib を利用するにとどめ、メインループの制御、フレームレート固定化、メモリ空間の確保と破棄、オブジェクトの生存周期管理に至るまで自作のアーキテクチャを採用しています。

単一責任の原則と疎結合設計： 各システムは独立したマネージャーオブジェクトとして分離されており、直接の相互参照を極力排除してイベント通知および中央管理機構を経由する設計をとることで、メモリ漏れの防止と安全な拡張性を確保しています。

シーンマネージャーおよびゲーム制御

タイトル、ステージ選択、メインゲーム、リザルトの各状態を管理。遷移時のリソース読み込みと解放を非同期的に分離し、スムーズな画面切り替えを実現します。

オブジェクトマネージャー

自機、バリア、敵キャラクター、および画面上に滞留する多大な数のオブジェクトの生成、更新、描画、消滅周期を一括して効率的に制御します。

エネミーマネージャー

敵のウェーブ出現シーケンスやボスの出現タイミング、段階ごとの行動フェーズ移行を管理し、ゲーム展開のスムーズな進行を保証します。

バレットマネージャー

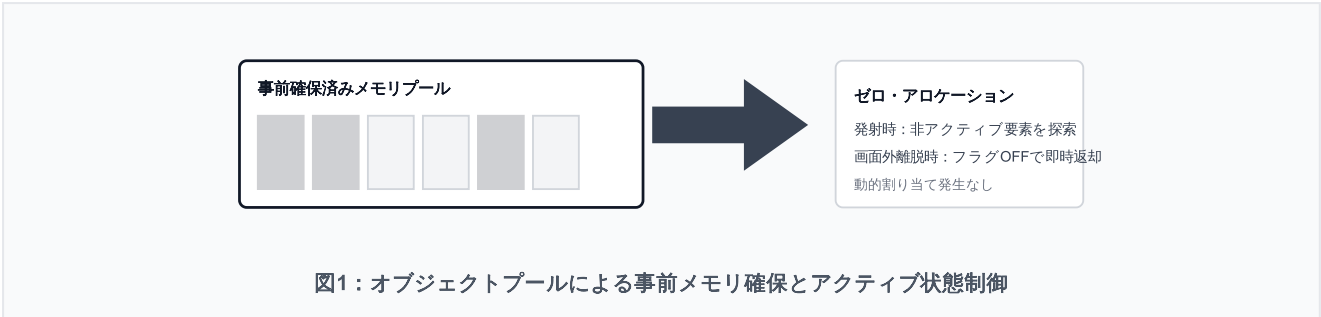
全弾幕オブジェクトの再利用管理と描画順序を最適化。オブジェクトプールを内部に保有し、メモリ割り当てコストを最小化します。

3. 注力した技術ポイント1 弾幕パターン生成とオブジェクトプール

弾幕シューティングにおける最大の技術的課題は、画面上に数百から数千発の弾丸が同時に存在する際の動的メモリ割り当てオーバーヘッドの抑制と、数学的アプローチによる多様な弾幕パターンの計算処理です。

高速な描画とゼロ・アロケーションを実現するオブジェクトプール

ゲーム中に毎フレーム弾丸の生成と破棄を繰り返すと、ヒープ領域の断片化やガベージの発生による処理落ちが引き起こされます。本設計では、ゲーム開始時に十分な要素数の弾丸オブジェクトをアレイとして一括確保し、使用中フラグとリングバッファを用いて再利用するオブジェクトプール構造を自作しました。



三角関数および媒介変数を用いた幾何弾幕アルゴリズム

ボスの攻撃パターンにおいては、単調な全方位弾ではなく、美しい幾何学模様を描く弾幕の計算ロジックを極力軽い計算量で構築しています。

- **螺旋弾幕パラメータ制御**：フレームカウントを角度パラメータへ変換し、極座標系における半径と角度の増減量を動的に計算して発射ベクトルを決定。
- **波状・扇状拡散計算**：三角関数の周期性を利用し、弾丸群の初速にSin波を加算することで、画面内で波状に伸縮・うねりを生み出す変形弾幕を実現。
- **自機狙い誘導弾補正**：自機現在座標とボス発射口座標との差分ベクトルから逆正接関数を用いて角度を算出し、一定割合の追尾誤差を加えることで絶妙な難易度の狙い撃ち弾を生成。

4. 注力した技術ポイント2 ボスのステートパターンによる思考AI

ボスの体力が減少するにつれて、弾幕の密度や射出パターン、移動パターンが段階的に変化する多段フェーズ構造を整理されたコードで保守するため、ステートパターンを採用しました。

状態遷移の独立カプセル化	安全な無敵時間と必殺技制御
ボスの行動基底クラスから、通常弾幕ステート、狂乱弾幕ステート、必殺技準備ステートなどへ派生。各ステートクラスが自身の更新処理と遷移条件の判定を自己完結して保持します。	フェーズ切り替え時には一時的な無敵状態ステートへ強制遷移させ、弾幕消去と同時に次フェーズの準備動作へ安全に接続することで、連続ダメージによるバグや予期せぬ破壊を防ぎます。

5. 注力した技術ポイント3 データ駆動設計と演出システム

プログラムの再ビルドを行うことなく、ゲームバランスの調整やアセットの変更を柔軟に行える設計を目指し、外部ファイルによるデータ駆動設計を導入しました。

外部ファイルによるアセットおよびパラメータ管理

画像ファイルや音声データのパス、敵の体力や出現条件、弾幕の発射間隔などの数値をCSV外部ファイルとして切り離して管理しています。リソースマネージャーが起動時にこれらを読み込み、文字列識別子をキーとしたキャッシュ管理を行うことで、アセット追加や調整時にコード修正を不要にしました。

手応えを高めるゲーム演出のプログラミング

画面揺れ演出	ヒットストップ	ダメージフラッシュ
ボス撃破時や大ダメージ時に描画カメラの原点座標に微小なノイズを加えることで迫力ある振動を表現。	スペルカード命中時にゲーム全体の更新処理を数フレーム意図的に停止させ、打撃の手応えを強調。	自機被弾時に画面全体へ白のアルファ値を瞬間的に合成し、プレイヤーへ危機状況を直感的に伝達。

6. 課題解決と開発プロセス

生成補助コード導入時の不具合対策と管理徹底

開発過程においてコード作成補助ツールを利用した際、既存の変数が意図せず変更され、弾幕の発射ロジックや動作制御が意図通り機能しなくなる課題が発生しました。これに対し、コード変更時の影響範囲チェックと段階的な動作検証を厳格化しました。変更前後の構造比較と不具合原因の記録を徹底することで、迅速な原因特定と品質維持を行う開発プロセスを確立しました。

大量の弾幕発生時における処理負荷の最適化

画面上に数百発を超える弾丸が存在する場合、毎フレームの無駄な処理により処理速度が低下する恐れがあります。この問題を解決するため、以下の二段階による最適化処理を実装し、安定した処理速度を維持しました。

- **画面外領域における早期破棄処理**：画面枠から指定ピクセル以上外へ離脱した弾丸オブジェクトは、アクティブフラグを即座に解除して描画および更新処理をスキップ。
- **更新フラグによる処理の遮断**：画面外に待機している非アクティブ状態の弾丸はループの最上流でチェックを行い、無駄な座標計算や三角関数処理を行わないよう即座に通過。

7. 開発成果

本作品の制作を通じて、C++による手動メモリ管理、オブジェクト指向に基づくカプセル化や多態性の実践力を養いました。オブジェクトプールや弾幕生成エンジン、ステートマシンをサードパーティ製エンジンに頼らずゼロから自作したことにより、ゲームエンジンの内部で実行されている抽象的な処理の仕組みに対する深い洞察を得ることができました。